DHBW
Duale Hochschule
Baden-Württemberg

Cheatsheet Programmierung II

Daniel Appenmaier, 26. Juli 2026

Cheatsheet

java.lang

```
Boolean - int compare(x: boolean, y: boolean) //static
Boolean - Boolean valueOf(s: String) //static
Boolean - Boolean valueOf(b: boolean) //static
Comparable<T> - int compareTo(o: T)
Double - int compare(d1: double, d2: double) //static
Double - Double valueOf(s: String) //static
Double - Double valueOf(d: double) //static
Integer - int compare(x: int, y: int) //static
Integer - Integer valueOf(s: String) //static
Integer - Integer valueOf(i: int) //static
Long - long compare(x: long, y: long) //static
Long - Long valueOf(s: String) //static
Long - Long valueOf(l: long) //static
Object - boolean equals(object: Object)
Object - int hashCode()
Object - String toString()
Runnable - void run()
String - char charAt(index: int)
String - int length()
String - String[] split(regex: String)
String - String toLowerCase()
String - String toUpperCase()
System - long currentTimeMillis() //static
```

java.time

```
LocalDate - int getDayOfMonth()
LocalDate - int getDayOfYear()
LocalDate - Month getMonth()
LocalDate - int getMonthValue()
LocalDate - int getYear()
LocalDate - LocalDate now() //static
LocalDate - LocalDate of(year: int, month: int, dayOfMonth: int) //static
LocalDate - LocalDate parse(text: CharSequence) //static
LocalTime - int getHour()
LocalTime - int getMinute()
LocalTime - int getSecond()
LocalTime - LocalTime now() //static
LocalTime - LocalTime of(hour: int, minute: int, second: int) //static
LocalTime - LocalTime parse(text: CharSequence) //static
```

java.io

```
PrintStream - void print(obj: Object)
PrintStream - PrintStream printf(String format, Object... args)
PrintStream - void println()
PrintStream - void println(x: Object)
```

java.util

```
Collections - void sort(list: List<T>) //static
Collections - void sort(list: List<T>, c: Comparator<T>) //static
Comparator<T> - int compare(o1: T, o2: T)
Comparator<T> - Comparator<T> comparing(keyExtractor: Function<T,U>) //static
Entry<K,V> - K getKey()
Entry<K,V> - V getValue()
Entry<K,V> - V setValue(value: V)
Enumeration - Enumeration valueOf(arg0: String) //static
Enumeration - Enumeration[] values() //static
List<E> - boolean add(e: E)
List<E> - void add(index: int, element: E)
List<E> - boolean contains(o: Object)
List<E> - void forEach(action: Consumer<T>)
List<E> - E get(index: int)
List<E> - List<E> of(elements: E...) //static
List<E> - E remove(index: int)
List<E> - boolean remove(o: Object)
List<E> - List<T> reversed()
List<E> - int size()
List<E> - void sort(c: Comparator<T>)
Map<K,V> - boolean containsKey(key: Object)
Map<K,V> - boolean containsValue(value: Object)
Map<K,V> - Set<Entry<K,V>> entrySet()
Map<K,V> - void forEach(action: BiConsumer<K,V>)
Map<K,V> - V get(key: Object)
Map<K,V> - Set<K> keySet()
Map<K,V> - V put(key: K, value: V)
Map<K,V> - V putIfAbsent(key: K, value: V)
Map<K,V> - Collection<V> values()
Optional<T> - Optional<T> empty() //static
Optional<T> - T get()
Optional<T> - void ifPresent(action: Consumer<T>)
Optional<T> - void ifPresentOrElse(action: Consumer<T>, emptyAction: Runnable)
Optional<T> - boolean isPresent()
Optional<T> - Optional<T> of(t: T) //static
Optional<T> - Optional<T> ofNullable(t: T) //static
Optional<T> - T orElse(other: T)
OptionalDouble - OptionalDouble empty() //static
OptionalDouble - double getAsDouble()
OptionalDouble - void ifPresent(action: DoubleConsumer)
OptionalDouble - void ifPresentOrElse(action: DoubleConsumer, emptyAction: Runnable)
OptionalDouble - boolean isPresent()
OptionalDouble - OptionalDouble of(value: double) //static
OptionalDouble - double orElse(other: double)
Random - int nextInt(origin: int, bound: int)
```

java.util.function

```
BiConsumer - void accept(t: T, u: U)
Consumer<T> - void accept(t: T)
DoubleConsumer - void accept(value: double)
Function<T,R> - R apply(t: T)
Predicate<T> - boolean test(t: T)
ToDoubleFunction<T,R> - double applyAsDouble(value: T)
ToIntFunction<T,R> - int applyAsInt(value: T)
```

java.util.stream

```
Collectors - Collector<T,?,Map<K,List<T>>> groupingBy(classifier: Function<T,K>) //static
DoubleStream - OptionalDouble average()
DoubleStream - double sum()
IntStream - OptionalDouble average()
IntStream - int sum()
Stream<T> - boolean allMatch(predicate: Predicate<T>)
Stream<T> - boolean anyMatch(predicate: Predicate<T>)
Stream<T> - R collect(collector: Collector<T,A,R>)
Stream<T> - long count()
Stream<T> - Stream<T> distinct()
Stream<T> - Stream<T> filter(predicate: Predicate<T>)
Stream<T> - Optional<T> findAny()
Stream<T> - Optional<T> findFirst()
Stream<T> - void forEach(action: Consumer<T>)
Stream<T> - Stream<T> limit(maxSize: long)
Stream<T> - Stream<R> map(mapper: Function<T,R>)
Stream<T> - DoubleStream mapToDouble(mapper: ToDoubleFunction<T,R>)
Stream<T> - IntStream mapToInt(mapper: ToIntFunction<T,R>)
Stream<T> - Optional<T> max(comparator: Comparator<T>)
Stream<T> - Optional<T> min(comparator: Comparator<T>)
Stream<T> - boolean noneMatch(predicate: Predicate<T>)
Stream<T> - Stream<T> skip(n: long)
Stream<T> - Stream<T> sorted()
Stream<T> - Stream<T> sorted(comparator: Comparator<T>)
Stream<T> - List<T> toList()
```

org.junit.jupiter.api

```
Assertions - void assertEquals(expected: Object, actual: Object) //static
Assertions - void assertFalse(condition: boolean) //static
Assertions - void assertNotEquals(expected: Object, actual: Object) //static
Assertions - void assertNotNull(actual: Object) //static
Assertions - void assertNotSame(expected: Object, actual: Object) //static
Assertions - void assertNull(actual: Object) //static
Assertions - void assertSame(expected: Object, actual: Object) //static
Assertions - T assertThrows(expectedType: Class<T>, executable: Executable) //static
Assertions - void assertTrue(condition: boolean) //static
Executable - void execute()
```

org.mockito / org.mockito.stubbing

```
Mockito - OngoingStubbing<T> when(methodCall: T) //static
MockitoAnnotations - AutoCloseable openMocks(testClass: Object) //static
OngoingStubbing<T> - OngoingStubbing<T> thenReturn(value: T)
```